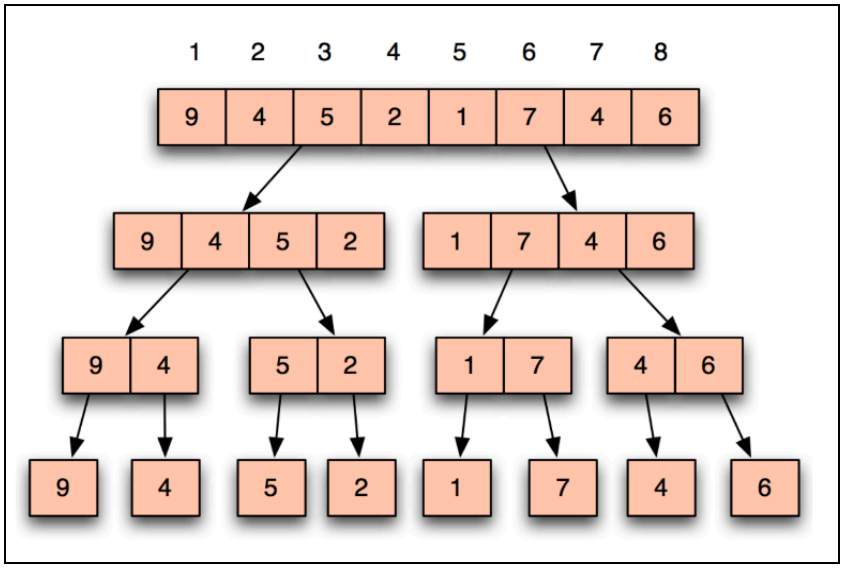
Sorting Algorithms

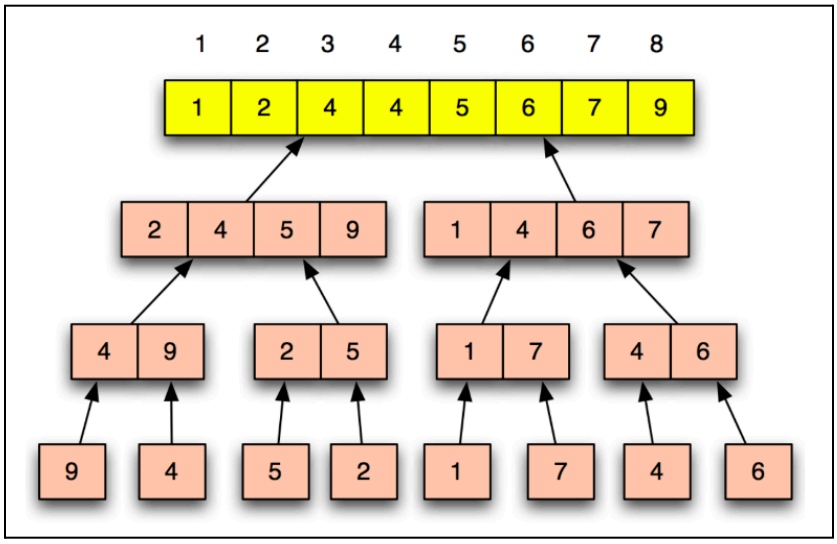
Merge Sort:

This sort follows a divide-and-conquer algorithm. It divides the problem into half, recursively sorts two subproblems, and then merges the results into a complete sorted sequence.

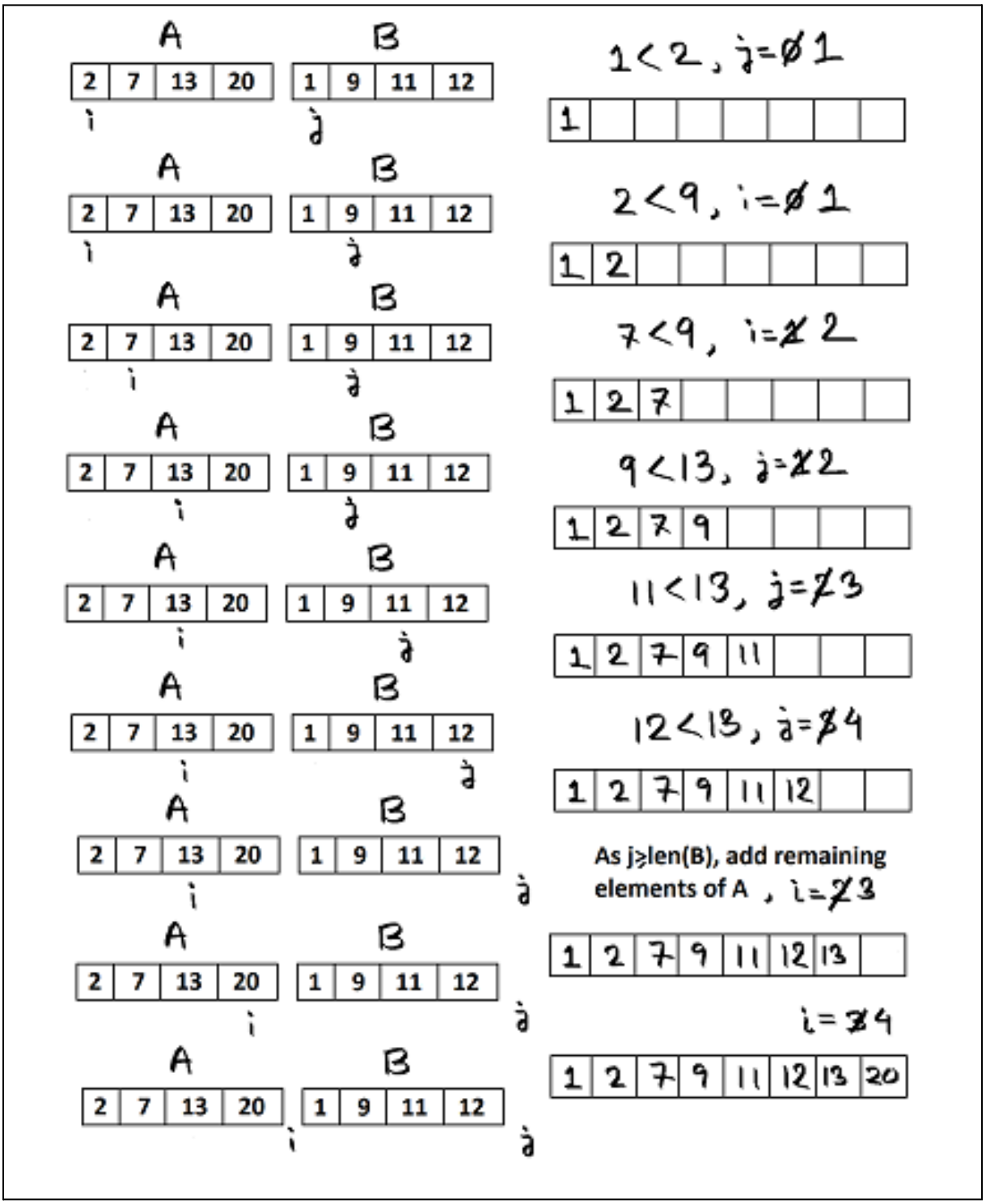
Divide:



Conquer & Combine:



Merging/Combining two sorted arrays A and B into another sorted array C in Θ(n):



Pseudo Code:

```
MERGE-SORT(A) ▷ A[0 : n]
1  if n = 1
2     then return
3     else                          ▷ recursively sort the two subarrays
4          A_1 = MERGE-SORT(A[0 : n//2])
5          A_2 = MERGE-SORT(A[ n//2 ] + 1 : n])
6          A = MERGE(A_1, A_2)          ▷ merge the sorted arrays

MERGE(A, B)
   INPUT: Two sorted arrays A and B
   OUTPUT: Returns C as the merged array
   ▷ n_1 = length[A], n_2 = length[B], n = n_1 + n_2
1  Create C of length n
2  Initialize two indices i and j to point to A and B respectively
3  while i < len(A) and j < len(B)
4        do Select the smaller of two and add to end of C
5           Advance the index that points to the smaller one
6  if i < len(A) or j < len(B)
7     then Copy the rest of the non-empty array to the end of C
8  return C
```

Advantages:

- Consistent time complexity O(n log n), making it efficient for large datasets
- Stable sorting algorithm, preserving the relative order of equal elements

Disadvantages:

- Requires additional memory space (Out-of-place sorting) for the temporary arrays during the merging process, leading to higher space complexity O(n)
- Slower for small datasets compared to simpler algorithms like insertion sort

Follow-up Question:

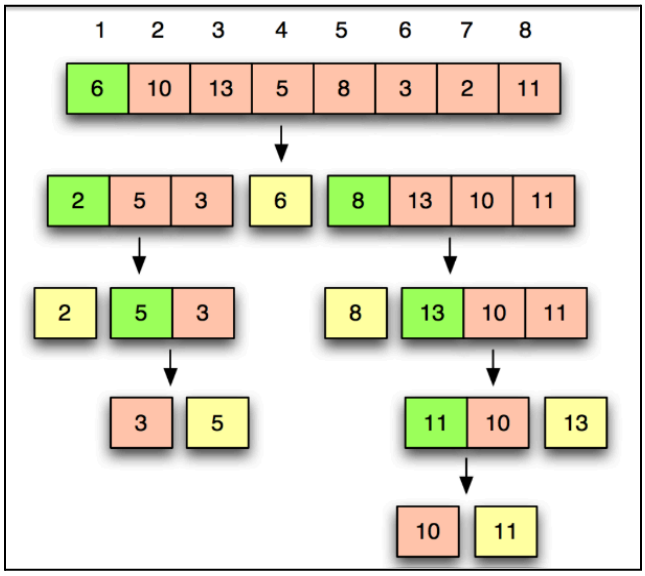
Q) What is the Recurrence Equation?

Ans: It depends on the number of subproblems, each subproblem size, and the work done at each step. In each step, there are 2 subproblems where size gets divided by 2 (n/2) and work done is n at each step. T(n) = 2T(n/2) + n

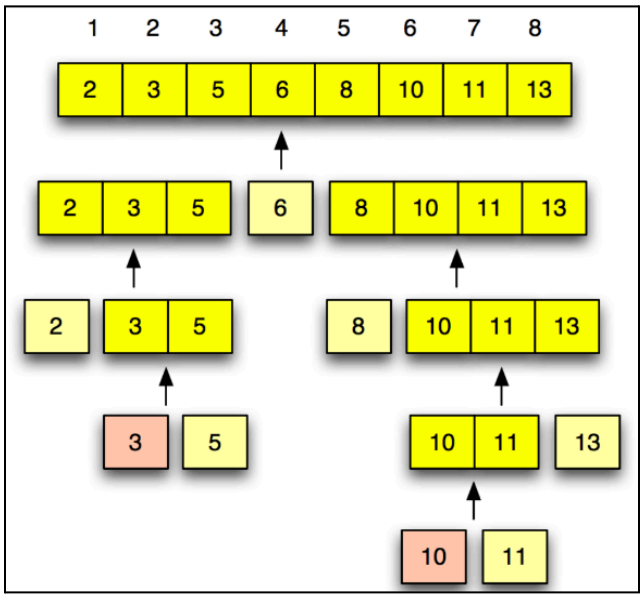
Quick Sort:

This sort also follows a divide-and-conquer algorithm. It partitions the array into subarrays around a pivot x such that the elements in the lower subarray ≤ x ≤ elements in the upper subarray, recursively sort the 2 subarrays, and then concatenates the lower subarray, pivot, and the upper subarray.

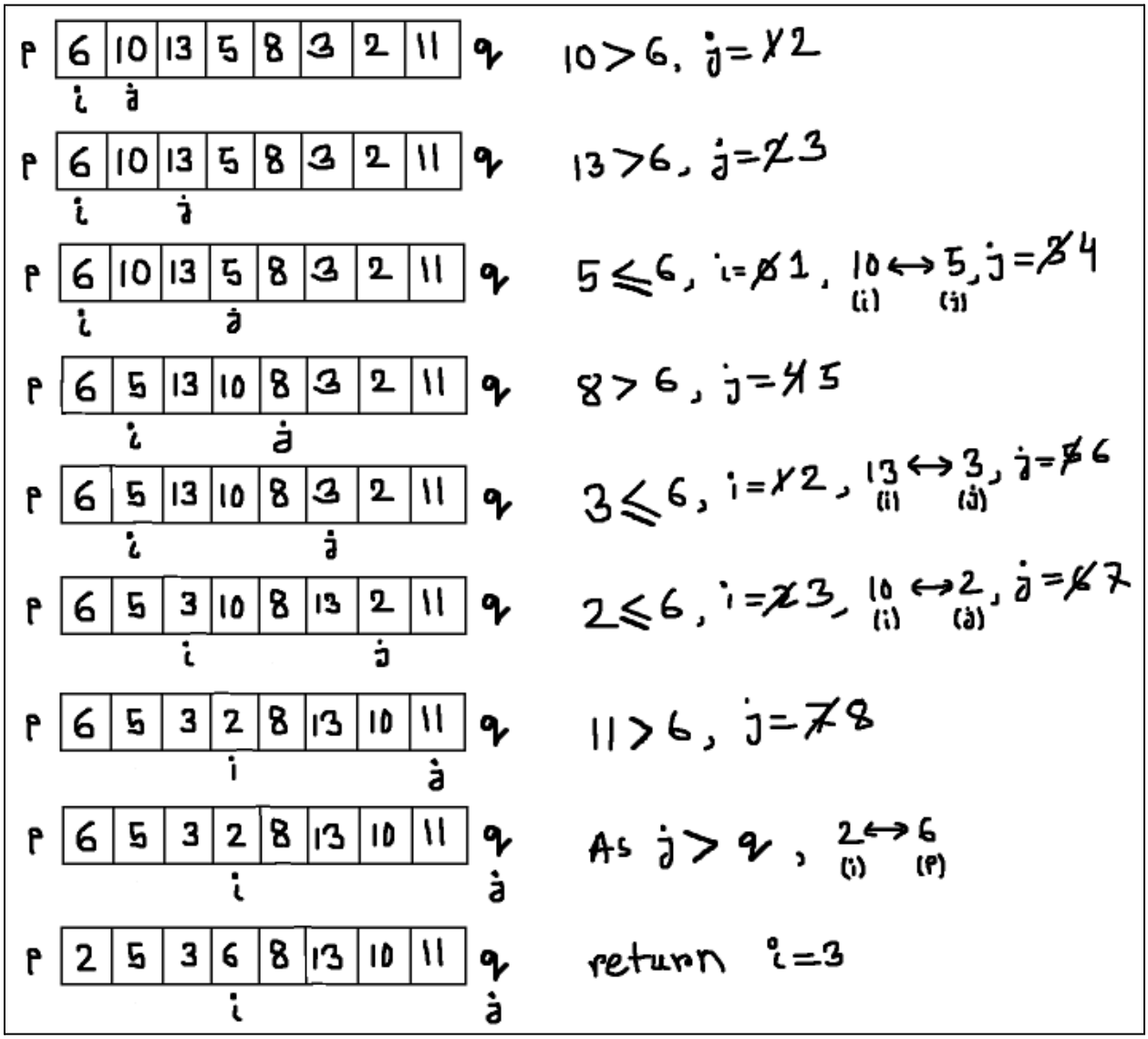
Divide / Partition:



Conquer & Combine:



Partitioning/Dividing an array A in Θ(n):



Pseudo Code:

```
QUICKSORT(A, l, r) ▷ A[0 : n]
1  if l < r
2     then pivot ← PARTITION(A, l, r)
3          QUICKSORT(A, l, pivot − 1)
4          QUICKSORT(A, pivot + 1, r)

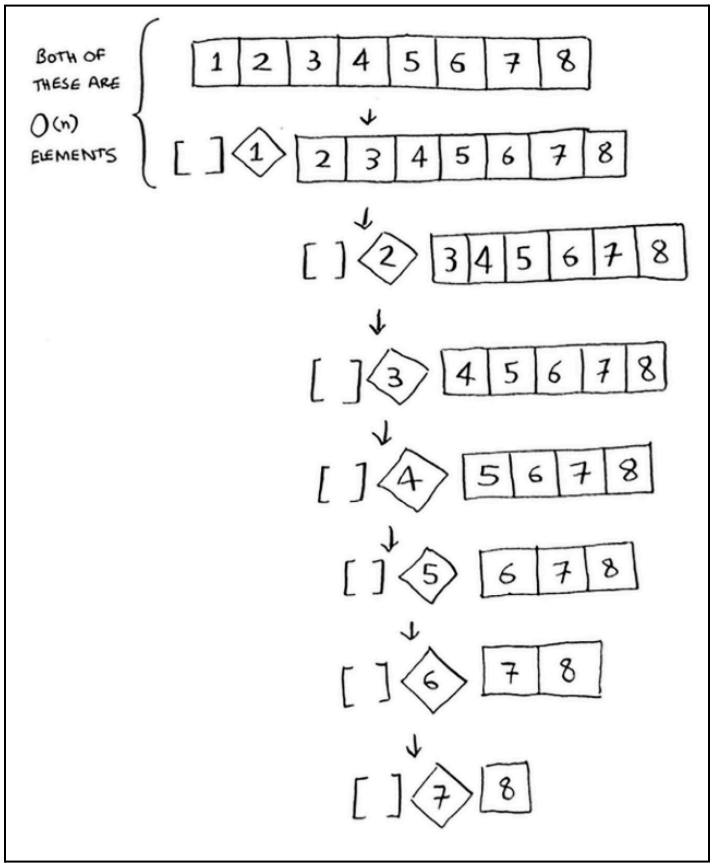
PARTITION(A, p, q) ▷ A[p : q]
1  x ← A[p]          ▷ pivot = A[p]
2  i ← p
3  for j ← p + 1 to q
4       do if A[j] ≤ x
5             then i ← i + 1
6                  exchange A[i] ↔ A[j]
7  exchange A[p] ↔ A[i]
8  return i
```

Quicksort is unique because its speed depends on the pivot you choose.

Worst Case vs Average/Best Case:

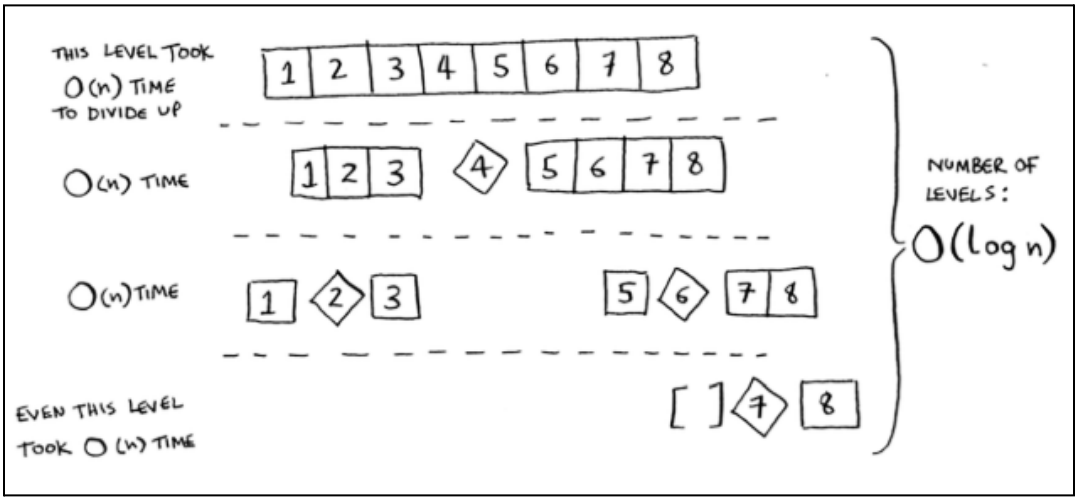
Worst Case happens when the pivot is the first or last element of a sorted (ascending or descending) array. The result is that one of the partitions is always empty.

Worst Case:



There are O(n) levels or the height is O(n), And each level takes O(n) time. The entire algorithm will take O(n) * O(n) = $O(n^2)$ time.

Best Case:



There are O(log n) levels or the height is O(log n). And each level takes O(n) time. The entire algorithm will take O(n) * O(log n) = O(n log n) time.

The best case is also the average case. If you always choose a random element in the array as the pivot, quicksort will complete in O(n log n) time on average

Advantages:

- Fastest practical sorting algorithm O(n log n), making it efficient for large datasets
- Does not require additional memory space (In-place sorting)

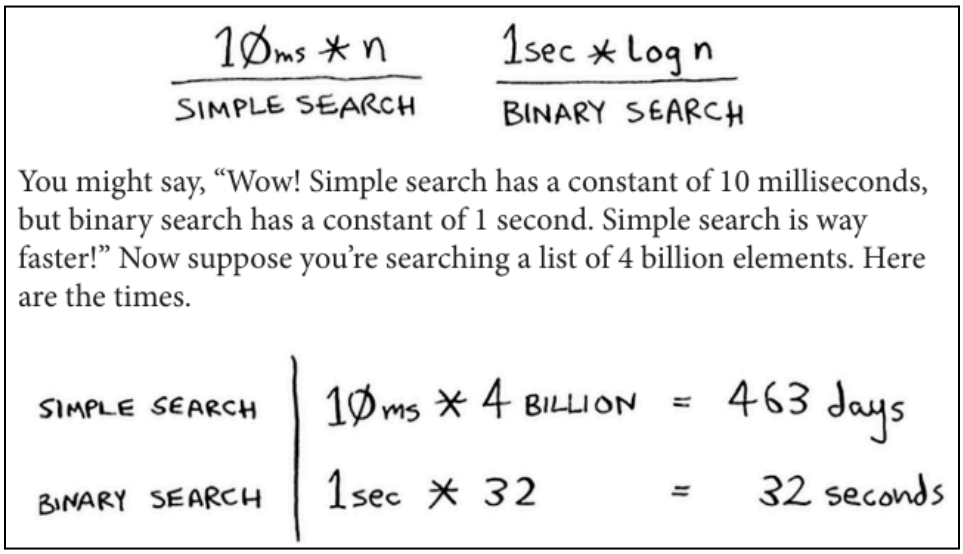
Disadvantages:

- $O(n^2)$ time complexity in worst-case scenario
- Unstable sorting algorithm, not preserving the relative order of equal elements

Follow-up Question:

Q) If quicksort is O(n log n) on average, but merge sort is O(n log n) always, why not use merge sort? Isn't it faster?

Ans: Even though both functions are the same speed in Big O notation, quicksort is faster in practice. When you write Big O notation like O(n), it means O(c * n) where c is some fixed amount of time that your algorithm takes. Let's see an example:



Quicksort has a smaller constant than merge sort. So if they're both O(n log n) time, quicksort is faster. And quicksort is faster in practice because it hits the average case way more often than the worst case.

Q) What is the Recurrence Equation when it is worst-case?

Ans: It depends on the number of subproblems, each subproblem size, and the work done at each step. In each step, there is 1 subproblem where size gets subtracted by 1 (n-1) and work done is n at each step. T(n) = T(n-1) + n

Q) What is the Recurrence Equation when it is best-case?

Ans: It depends on the number of subproblems, each subproblem size, and the work done at each step. In each step, there are 2 subproblems where size gets divided by 2 (n/2) and work done is n at each step. T(n) = 2T(n/2) + n

Q) How do you make sure your algorithm never reaches the worst-case when choosing 1st element as a pivot?

Ans: Use Randomized Quicksort. It is the same as the usual Quicksort but you will swap the pivot with a random number in the array within the range and then start partitioning. This will always give O(n log n).

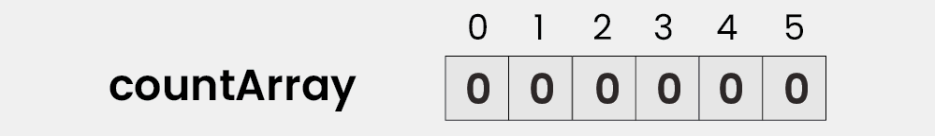
Count Sort:

Count sort, also known as counting sort, is a non-comparative integer sorting algorithm. This sorting technique does not perform sorting by comparing elements but by using a frequency array. It is efficient when the range of the input data, k (i.e., the difference between the maximum and minimum values) is not significantly greater than the number of elements to be sorted.

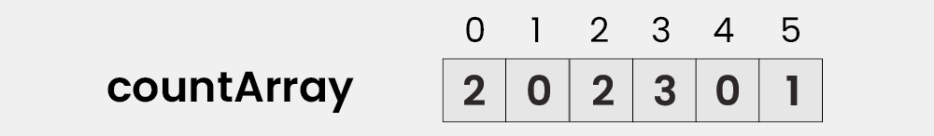
Step 1: Find out the range k (max+1) from the given array.



Step 2: Initialize a countArray[] of length k (5+1) with all elements as 0. This array will be used to store the occurrences of the elements of the input array.



Step 3: In the countArray[], store the count of each unique element of the input array at their respective indices.

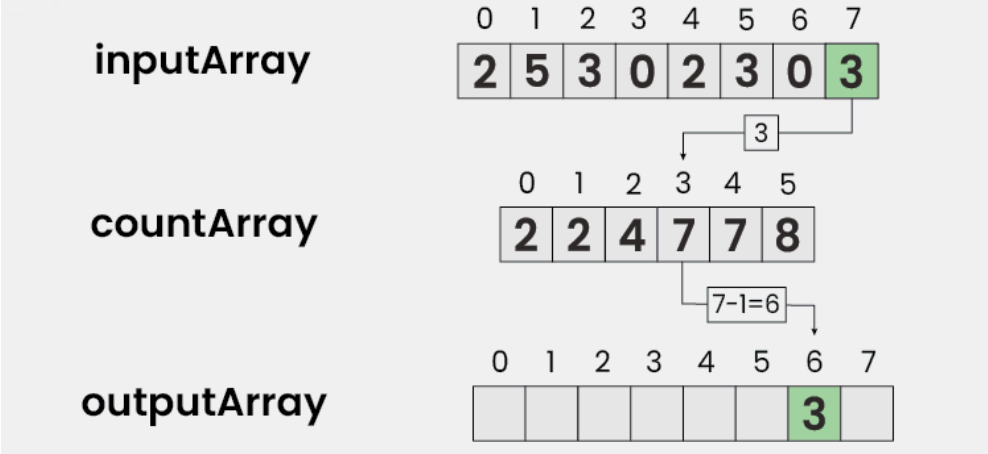


Step 4: Store the cumulative sum of the elements of the countArray[] by doing countArray[i] = countArray[i – 1] + countArray[i].

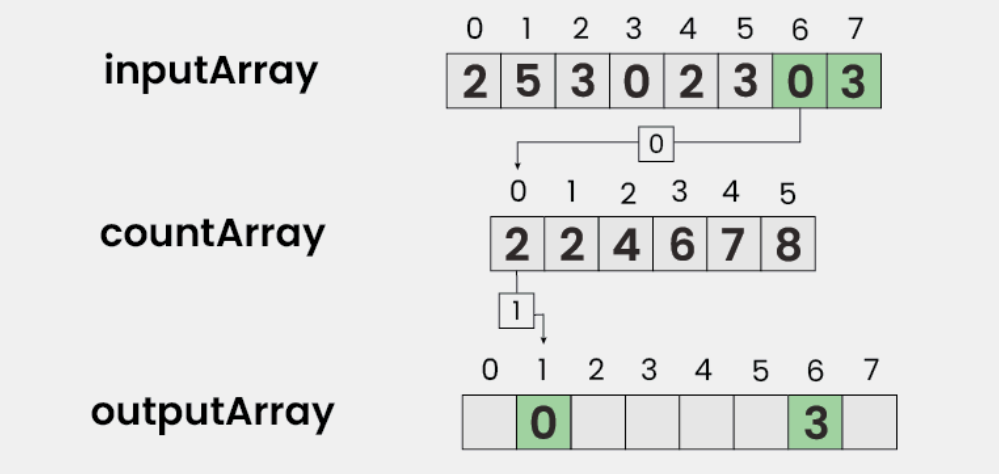


Step 5: Iterate from the end of the input array (to preserve stability) and update outputArray[countArray[inputArray[i]] – 1] = inputArray[i] and also, update countArray[inputArray[i]] = countArray[inputArray[i]] – 1 (so that duplicate values are not overwritten)

For i=7

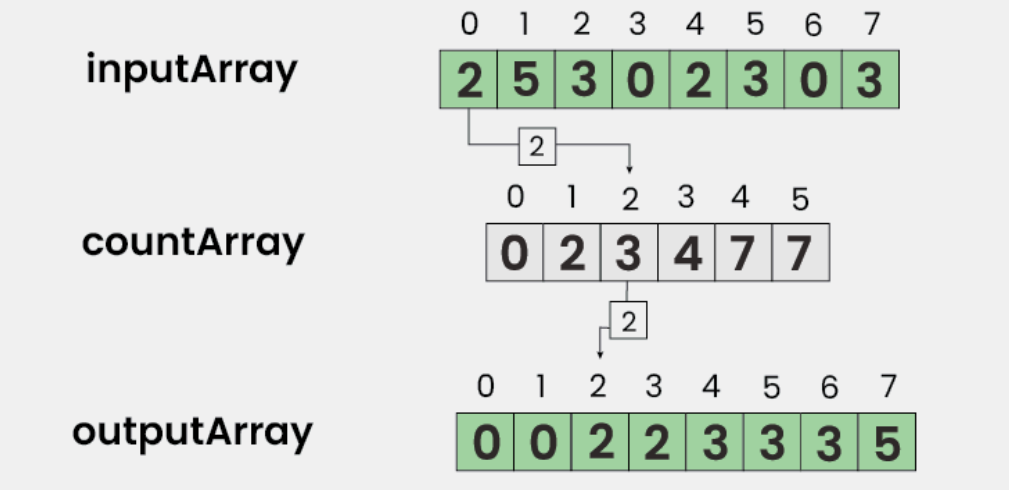


For i=6



.
.
.

For i=0



The outputArray will return a sorted array

Pseudo Code:

```
Counting-Sort(A,B,k)
    for i ←0 to k
        do C[i] ←0
    for j ←0 to length[A]-1
        do C[A[j]] ←C[A[j]]+1
    // C[i] contains number of elements equal to i.
    for i ←1 to k
        do C[i]=C[i]+C[i-1]
    // C[i] contains number of elements ≤ i.
    for j ←length[A]-1 downto 0
        do B[C[A[j]]-1] ←A[j]
           C[A[j]] ←C[A[j]]-1
```

Advantages:

- Consistent linear time complexity O(n+k)
- Stable sorting algorithm, preserving the relative order of equal elements

Disadvantages:

- Counting sort is inefficient if the range of values to be sorted is very large
- Requires additional memory space (Out-of-place sorting) for countArray and outputArray, leading to higher space complexity O(n+k)
- Counting sort does not directly work on decimal or negative values

Follow-Up Question:

Q) What is the Time Complexity?

Ans: The first loop takes O(k), the second loop takes O(n), the third loop takes O(k) and the last loop takes O(n). Hence, time complexity is O(n+k)